

ECE391

Computer System Engineering

Lecture 3



Prof. Kirill Levchenko and Dong Kai Wang

Slides adapted from Prof. Kalbarczyk

University of Illinois at Urbana- Champaign

Spring 2024

Administrivia

- Do not use lab room on Tuesdays **5-6PM** and 6-9PM.
 - Check the office hours calendar.
- Read ahead of lecture.
- Slides posted before lecture (but may differ slightly).
 - May be updated after lecture with annotations.

Lecture Topics

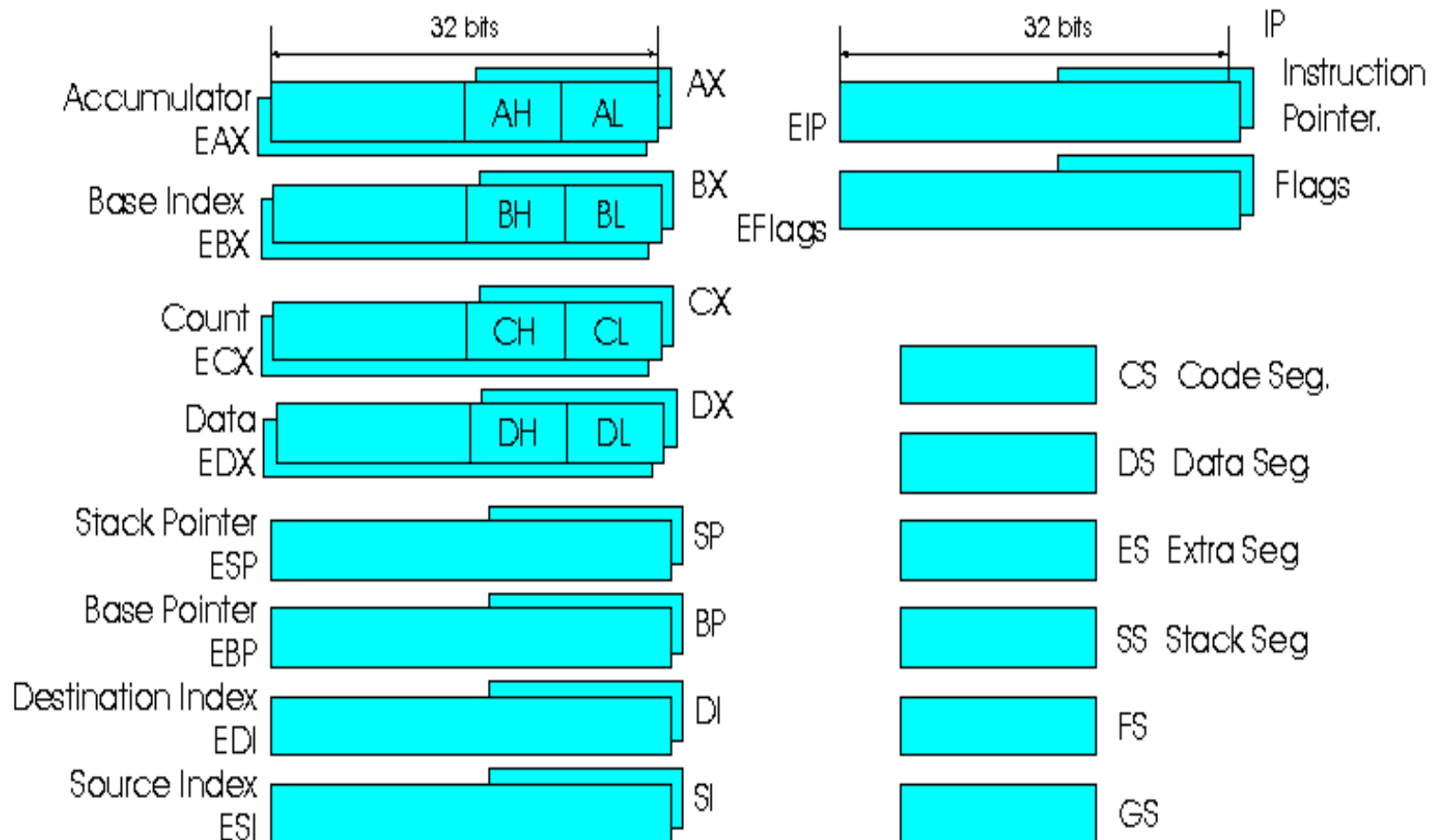
- Continue x86 instructions
- Conditional codes
- Control flow instructions
- Stack operations
- Calling convention

Quick x86 Review

- What is %CL?
- Is x86 big or little-endian?
- How many **general-purpose** registers in x86?
- Does %SP, %SH, %SL exist?

Recall: x86 Registers

General purpose registers



Condition Codes (in EFLAGS)

- Among others (not mentioned in this class)...
 - SF: sign flag: result is negative when viewed as 2's complement data type
 - ZF: zero flag: result is exactly zero
 - CF: carry flag: unsigned carry or borrow occurred
(or other, instruction-dependent meaning, e.g., on shifts, it is the last bit shifted out)
 - OF: overflow flag: 2's complement overflow
(and other instruction-dependent meanings)
 - PF: parity flag: even parity in result (even # of 1 bits)
(*question: what is the parity flag used for?*)

What Instructions Set Flags (condition codes)?

- Not all instructions set flags (e.g., MOV, LEA, NOT)
- Some instructions set some flags! (e.g., ROR, INC)

– As usual, check a reference if unsure.

Flags Affected

ZF and SF flags are updated based on the result.
are undefined.

- Use **CMP** or **TEST** to set flags:

CMP **L** %EAX, %EBX # flags $\leftarrow$ (EBX – EAX)

TEST **L** %EAX, %EBX # flags $\leftarrow$ (EBX AND EAX)

Which flags?

- Note that EBX does not change in either case
- What combinations of flags are needed for unsigned/signed relationships comparator?

Control Flow Instructions (1)

- Consider two three-bit values A and B; How to decide if $A < B$?

	#1	#2	#3	#4	#5	#6
A	010	010	010	110	110	110
B	<u>-000</u>	<u>-110</u>	<u>-111</u>	<u>-000</u>	<u>-011</u>	<u>-111</u>
C	010	100	011	110	011	111

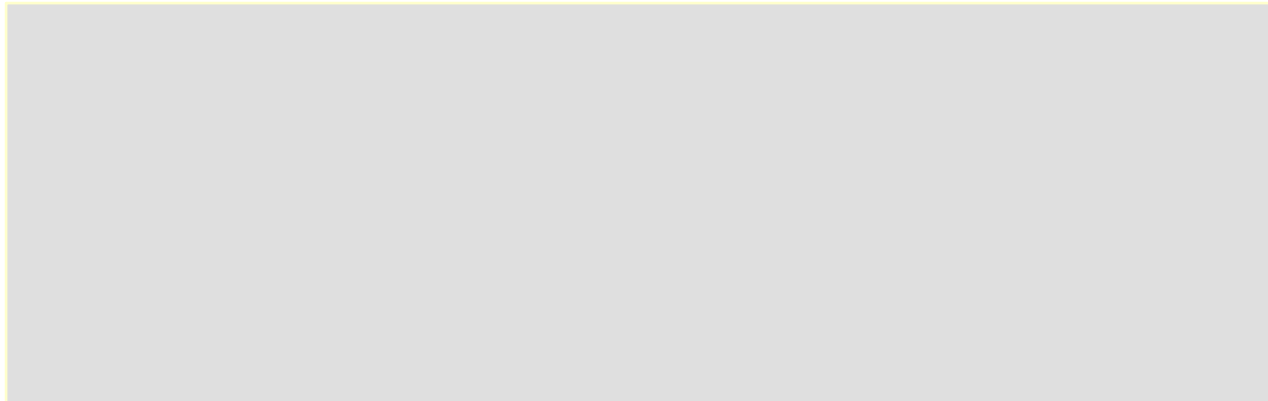
CF

OF

SF

unsigned <

signed <



Control Flow Instructions (1)

- Consider two three-bit values A and B; How to decide if $A < B$? (Spend ~5 minutes, compare with your neighbors)

	#1	#2 ₂	#3 ₂	#4	#5 ₋₂	#6 ₋₂
A	010	010	010	110	110	110
B	<u>-000</u>	<u>-110</u> ⁻²	<u>-111</u> ⁻¹	<u>-000</u>	<u>-011</u> ³	<u>-111</u> ⁻¹
C	010	100	011	110	011	111

CF

OF

SF

unsigned <

signed <

Control Flow Instructions (1)

- Consider two three-bit values A and B; How to decide if $A < B$? (Spend ~5 minutes, compare with your neighbors)

	#1	#2 ₂	#3 ₂	#4	#5 ₋₂	#6 ₋₂
A	010	010	010	110	110	110
B	-000	-110 ⁻²	-111 ⁻¹	-000	-011 ³	-111 ⁻¹
C	010	100	011	110	011	111
CF	0	1	1	0	0	1
OF	0	1	0	0	1	0
SF	0	1	0	1	0	1
unsigned <	No	Yes	Yes	No	No	Yes
signed <	No	No	No	Yes	Yes	Yes

Control Flow Instructions (2)

- Note that CF suffices for unsigned <
- What about signed < ?

		CF/OF			
		00	01	11	10
SF	0	0	1	x	0
	1	1	x	0	1

- Answer: **OF XOR SF**
- As usual, check manual if unsure.

JL JNGE	Jump if less Jump if not greater or equal	signed	SF <> OF
JGE JNL	Jump if greater or equal Jump if not less	signed	SF = OF

Branch Mnemonics

- Unsigned comparisons: “above” and “below”
- Signed comparisons: “less” and “greater”
- Both: equal/zero

unsigned jne jb jbe je jae ja

relationship $\neq$ < $\leq$ = $\geq$ >

signed jne jl jle je jge jg

- Adding “n” after “j” negates the condition checked
- Forms shown are those used when disassembling
 - do not expect binary to retain your version
 - e.g., “jne” becomes “jb”

Other Control Instructions

- Other branches
 - `jo` — jump on overflow (OF)
 - `jp` — jump on parity (PF)
 - `js` — jump on sign (SF)
 - `jmp` — unconditional jump
- **Control instructions:** subroutine call and return

<code>CALL printf</code>	<code># (push EIP), EIP ← printf</code>	Q: which access mem?
<code>CALL *%EAX</code>	<code># (push EIP), EIP ← EAX</code>	
<code>CALL *(%EAX)</code>	<code># (push EIP), EIP ← M[EAX]</code>	

Note: Branch addressing using registers or memory operands must be prefixed by *

<code>RET</code>	<code># EIP ← M[ESP], ESP ← ESP + 4</code>
------------------	--

Stack Operations

- Push and pop supported directly by x86 ISA

PUSHL %EAX # $M[ESP - 4] \leftarrow EAX, ESP \leftarrow \underline{ESP - 4}$

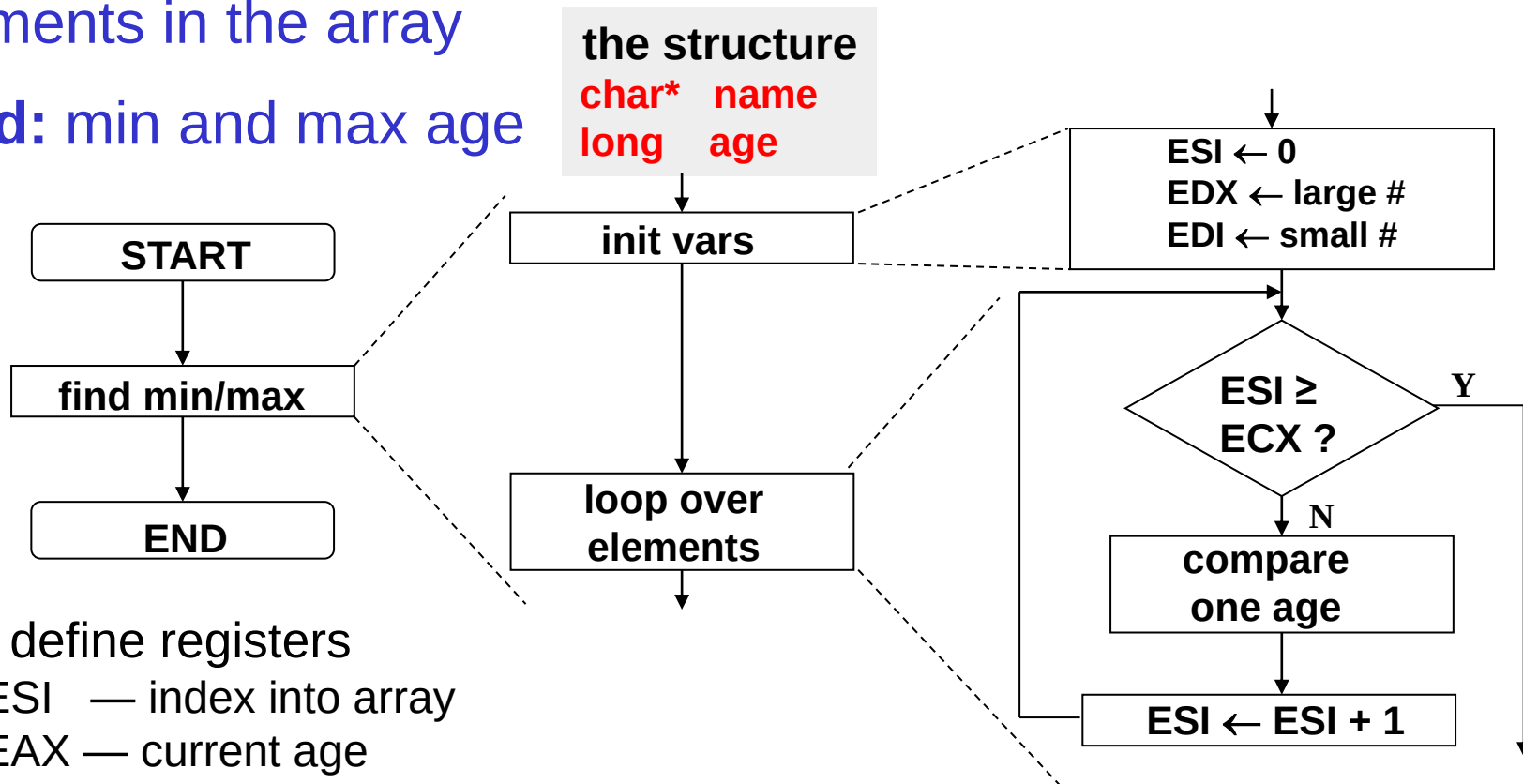
POPL %EBP # $EBP \leftarrow M[ESP], ESP \leftarrow ESP + 4$

PUSHFL

POPFL

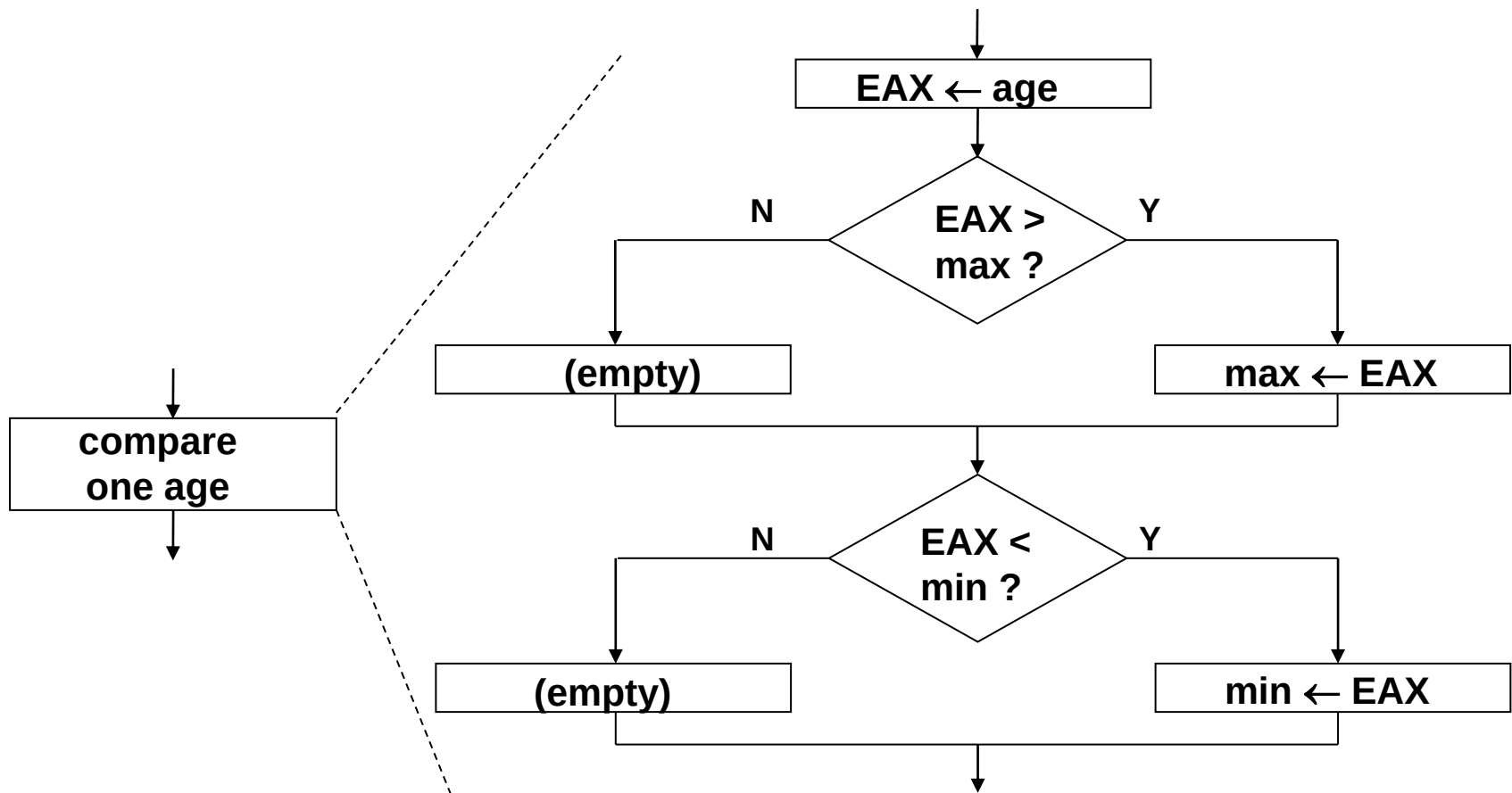
Code Example (Class Exercise)

- **Given:** EBX pointing to an array of structures with ECX elements in the array
- **Find:** min and max age



- first, define registers
 - ESI — index into array
 - EAX — current age
 - EDX — min age seen
 - EDI — max age seen
- next, use systematic decomposition...

Code Example - Loop Body



Code Example – Assembly (1)

```
XORL %ESI,%ESI          # init vars
```

```
MOVL TWO_MM,%EDX
```

```
MOVL TWO_MM+4,%EDI
```

LOOP:

```
CMPL %ECX,%ESI          # loop test; flags set based on (esi-ecx)
```

```
JGE     DONE
```

```
# read the age using one memory reference.
```

```
MOVL 4(%EBX,%ESI,8),%EAX
```

```
CMPL %EDI,%EAX          # check the max. age
```

```
JLE     NOT_MAX
```

```
MOVL %EAX,%EDI
```

Code Example – Assembly (2)

NOT_MAX:

```
    CMPL    %EDX,%EAX           # check the min age
    JGE     NOT_MIN
    MOVL    %EAX,%EDX
```

NOT_MIN:

```
    INCL    %ESI                # loop update
    JMP     LOOP
```

DONE: # more code ...

TWO_MM: .LONG 0x7FFFFFFF, 0x80000000

The Calling Convention (1)

- What is a calling convention?
 - generally: rules for subroutine interface structure
 - specifically
 - how information is passed into subroutine
 - how information is returned to caller
 - who owns registers
 - often specified by vendor so that different compilers' code can work together (it's a CONVENTION)
- Parameters for subroutines
 - pushed onto stack
 - from right to left in C
 - order can be language-dependent

The Calling Convention (2)

- Subroutine return values
 - EAX for up to 32 bits
 - EDX:EAX for up to 64 bits
 - floating-point not discussed
- Register ownership
 - return values can be clobbered by subroutine: EAX and EDX
 - caller-saved: subroutine free to clobber; caller must preserve
 - ECX
 - EFLAGS
 - callee-saved: subroutine must preserve value passed in
 - stack structure: ESP and EBP
 - other registers: EBX, ESI, and EDI

Stack Frames in x86 (1)

- The call sequence
 0. save caller-saved registers (if desired)
 1. **push** arguments onto stack
 2. make the call
 3. **pop** arguments off the stack
 4. restore caller-saved registers

Stack Frames in x86 (2)

- The **callee sequence** (creates the stack frame)
 0. save old base pointer and get new one
 1. save callee-saved registers (always)
 2. make space for local variables
 3. do the function body
 4. tear down stack frame (locals)
 5. restore callee-saved registers
 6. load old base pointer
 7. return